

Document Chunk Visualization

Highlighted words are overlap copied from the previous chunk.

Chunk 1

transformer-rag-guide-recursive-001

Strategy recursive
Words 72
Overlap 0
Pages 1-1
Sections Introduction to Transformers, Self-Attention

Introduction to Transformers Transformers process sequences using attention instead of recurrent hidden states. This design lets all tokens be processed in parallel during training. The architecture has become important for language models, vision systems, and multimodal applications. Self-Attention Self-attention lets each token examine other tokens in the same sequence. Queries represent what a token is looking for. Keys describe what each token offers, and values carry the information that will be combined.

Chunk 2

transformer-rag-guide-recursive-002

Strategy recursive
Words 65
Overlap 15
Pages 1-1
Sections Self-Attention

Keys describe what each token offers, and values carry the information that will be combined. Attention weights decide how strongly the model should use each value. A sentence can contain relationships between words that are far apart. Self-attention creates direct paths between those words, which helps the model capture long-range dependencies. Multi-head attention repeats this process in parallel so different heads can learn different relationships.

Chunk 3

transformer-rag-guide-recursive-003

Strategy recursive
Words 63
Overlap 15
Pages 1-1
Sections Self-Attention, Embeddings

dependencies. Multi-head attention repeats this process in parallel so different heads can learn different relationships. Embeddings Before attention can operate, text is tokenized and converted into numeric identifiers. An embedding layer maps every identifier to a dense vector. Similar language patterns can learn related vector representations during training. Positional information is then added because attention alone does not know the order of tokens.

Chunk 4

transformer-rag-guide-recursive-004

Strategy recursive
Words 55
Overlap 15
Pages 1-2
Sections Embeddings, Retrieval-Augmented Generation

Positional information is then added because attention alone does not know the order of tokens. Retrieval-Augmented Generation Retrieval-Augmented Generation retrieves external knowledge before generating an answer. A user query is embedded, compared with stored vectors, and matched to the most relevant document chunks. The retrieved text is added to the model prompt as grounding context.

Chunk 5

transformer-rag-guide-recursive-005

Strategy recursive
Words 58
Overlap 15
Pages 2-2
Sections Retrieval-Augmented Generation

relevant document chunks. The retrieved text is added to the model prompt as grounding context. Embedding an entire long document as one vector can hide a small relevant section. Chunking solves this problem by creating several focused retrieval units. A question about self-attention can then retrieve the self-attention section instead of an unrelated discussion about deployment or training.

Chunk 6

transformer-rag-guide-recursive-006

Strategy recursive
Words 54
Overlap 15
Pages 2-2
Sections Retrieval-Augmented Generation, Chunk Size and Overlap

can then retrieve the self-attention section instead of an unrelated discussion about deployment or training. Chunk Size and Overlap Small chunks are precise but may lose surrounding explanation. Large chunks contain more context but may combine unrelated ideas. The best size depends on the document style, embedding model, user questions, and available context window.

Chunk 7

transformer-rag-guide-recursive-007

Strategy recursive
Words 55
Overlap 15
Pages 2-2
Sections Chunk Size and Overlap

best size depends on the document style, embedding model, user questions, and available context window. Overlap copies a small amount of text from one chunk into the next. This helps when an important definition begins near the end of one boundary and continues after it. Too much overlap wastes storage and can return repetitive results.

Chunk 8

transformer-rag-guide-recursive-008

Strategy recursive
Words 75
Overlap 15
Pages 2-2
Sections Chunk Size and Overlap, Metadata and Citations

boundary and continues after it. Too much overlap wastes storage and can return repetitive results. Metadata and Citations Every chunk should retain its source document, page range, section name, and stable chunk identifier. Metadata enables filtering, debugging, and citations. A production pipeline may also record timestamps, permissions, language, document type, and checksum information. Well-designed chunking improves retrieval quality before any language model is called. It is therefore both a data-engineering decision and a model-quality decision.